



AI Model Delegation Standard

Ring-fencing top-tier tokens - Customapp engineering

Version 1.1 | 12 July 2026

1. Why this standard

Top-tier models such as Fable and Opus are priced at a multiple of Sonnet-class models, token for token. That fact is easy to state and easy to forget mid-build, when the fastest thing to do is keep typing in whichever session happens to be open.

The AI Model Delegation Standard turns that instinct into a rule: the expensive model is an architect, not a typist. It writes briefs, reviews seams, and makes the calls that need judgement; the legwork - the file-by-file implementation, the reconnaissance, the documentation - runs on Sonnet, dispatched through the `builder` and `recon` subagents. The target is for top-tier usage to settle at ten to fifteen percent of a developer's total AI tokens in a given week: not zero, because architecture and security review are exactly where the expensive model earns its keep, but not the default either.

The case study proves the split, not just the theory. On the claim-report pipeline build (12 July 2026), four Sonnet builder agents did the implementation legwork - roughly 810,000 tokens across the queue migration, the worker, the render Lambda, and the documentation. The Fable session that specified, spot-checked, and verified that work spent roughly a tenth of that figure on the parts only a top-tier model should do: writing the fixed contracts, reviewing the seams where the parallel agents' work joined, fixing the one real security defect, and signing off. Worked through, that split lands under ten percent of total tokens on the top tier - comfortably inside the target, on a real change that landed about 3,400 lines in a single session.

2. The split

Every delegation decision comes down to matching the task to the cheapest tier that can do it without a quality loss. Three tiers, the same in every repo:

Tier	Does	Does not
Opus 5 (architect - the default top tier; Fable 5 only by named exception, at ~2x the price)	• Exploration synthesis • Architecture decisions • Writing briefs • Seam and security review • Debugging weird failures • Deploy go/no-go • User-facing narrative	• Type implementations • Re-read agent output wholesale • Mechanical edits
Sonnet 5 (<code>builder</code> , <code>recon</code> in <code>.claude/agents/</code>)	• Implementation from a brief • Fan-out recon • Docs from a spec • Self-verification	• Renegotiate contracts • Explore beyond the read-first list • Commit or deploy
Haiku	Genuinely mechanical transforms only	Anything touching a repo's conventions

Default every session to Sonnet; escalate to Fable or Opus deliberately. The `builder` and `recon` subagents are pinned to Sonnet in `.claude/agents/` in every repo, so dispatching to them from a Fable session cannot leak top-tier tokens into legwork - including exploration, which otherwise inherits the session's model and burns top-tier tokens just reading files.

3. How it works day to day

1. **Sessions default to Sonnet.** That is the repo-level model pin, not something to remember - open a repo and you are already on the cheap tier.
2. **Escalate deliberately with `/model`** up to Fable or Opus, and only for: architecture decisions, security-adjacent changes, cross-component design, and debugging once you are genuinely stuck - not from the first error.
3. **On a top-tier session, you write briefs and review seams.** You do not type implementations. If the next step is "now go build it", that is a brief for the `builder` subagent, not a task for the session you are sitting in.
4. **The subagents are already there.** `builder` (implementation from a brief) and `recon` (read-only fan-out exploration - file:line evidence, explicit notes on what it did not find, never a file dump) live in every repo's `.claude/agents/`, pinned to Sonnet - dispatching to them cannot accidentally burn top-tier tokens on reading files or typing code.
5. **Drop back down once the judgement call is made.** Brief written, seams reviewed, security call made - return to Sonnet for the next normal task rather than staying on the expensive model out of momentum.

4. Anatomy of a brief

A brief is the unit of delegation. Every one - written by a human or by an architect session - carries all six elements below. Skipping one is the most common way a delegation goes sideways: a vague read-first list costs far more than it saves, and a missing verification step means a failure can get reported as done.

1. **Context** - what and why in three to five sentences, including what other agents are building in parallel.
2. **Read-first list** - exact files and line ranges (for example, `index.ts lines 120-260`), nothing open-ended. This is the single biggest token lever: a vague brief makes the cheap model re-derive context, badly.
3. **FIXED contracts** - the interfaces between parallel agents, marked "do not renegotiate": request and response shapes, header names, column lists, status values.
4. **Deliverables with acceptance criteria** - enumerated files, each with what "done" means.
5. **Verify before finishing** - the exact commands the agent must run and pass before reporting: tests, type-check, the dash lint.
6. **Report cap and explicit don'ts** - "under 30 lines, deviations with reasons"; no commit, no deploy, no database writes unless the brief says so.

When a brief is silent on something small, the builder makes the smallest reasonable assumption and notes it. When it is silent on something load-bearing, the builder stops that part and reports the gap rather than guessing.

The skeleton below is the literal template. Copy it, fill in the angle brackets, delete nothing.

You are implementing <X> in repo <path> (branch checked out – work in place, do NOT commit).

Context

<3–5 sentences. What, why, what the parallel agents build, whose contracts are fixed.>

Read first (these exact files/ranges, nothing more)

– <file> lines <a–b> (<why>)

FIXED contracts (do not renegotiate)

– <interface>: <exact shape>

Deliverables

1. <file> – <what it must do; acceptance criteria>

Verify before finishing

– <command> passes

– unicode–dash scan on every touched file comes back clean (ASCII hyphens rule – use the

same grep as `npm run lint:dashes`; do not write the two dash characters into this brief)

Return: files created, verification output, key decisions, deviations with reasons.
Under 30 lines.

5. Worked example

THE CLAIM-REPORT PIPELINE BUILD - 12 JULY 2026

The team replaced a third-party MSSQL-based PDF and email report engine with an in-house pipeline: a render Lambda, a queue worker, the supporting migrations, and documentation. About 3,400 lines landed in a single session.

Three briefs, one contract each

The architect wrote fixed contracts before any code existed, then split the build three ways into disjoint directories: the queue migration, the worker, and the render Lambda. The worker agent coded against the migration's RPC before that migration file had actually landed, because the contract was the fixed point, not the file. All three composed cleanly. A fourth Sonnet agent handled the documentation.

Excerpt - one of the three parallel briefs (render Lambda):

```
You are implementing the claim-report render Lambda in jacana-portal (branch checked out - work in place, do NOT commit).
```

Context

```
This replaces a third-party report engine with an in-house PDF pipeline. Two other agents build the queue migration and the worker in parallel, in disjoint directories. This agent owns lambda/claim-report-render/ only.
```

FIXED contracts (do not renegotiate)

```
- POST Function URL, header X-Render-Key: <RENDER_SHARED_KEY>
- Body { html, file_name?, options? }; success 200 { success:true, pdf_base64, byte_length }
- Failure { success:false, error } - 400 bad html, 401 bad key, 503 key unset
```

Deliverables

```
1. lambda/claim-report-render/index.mjs - handler + renderHtmlToPdf()
```

Verify before finishing

```
- local render test passes; unicode-dash scan clean
```

```
Return: files created, verification output, deviations with reasons. Under 30 lines.
```

What came back

All three builders finished inside their briefs, but real-world facts intruded on each of them, and each one said so in its deviations section instead of silently working around it:

- AWS had started blocking new Node 20 Lambda creations; the agent shipped Node 22 instead, and flagged it.

- A PDF-rendering dependency had quietly raised its own minimum Node version within a patch release; the agent pinned the exact working version rather than guessing.
- A migration number the brief specified was already taken by other work landed in the meantime; the agent renumbered and flagged it rather than overwrite.

None of these were guesses. They were facts the brief could not have had, surfaced exactly where the standard says to look: the deviations section.

The seam review catch

The one genuine security defect in the whole build was in the architect's own brief: a grant that would have let anyone holding the public anon key redirect where claim-report emails get sent. The builder implemented it exactly as specified - and flagged the smell in its report rather than silently rewriting someone else's contract. Seam review caught it before deploy. The lesson generalises: review the seams, and review your own specs as if someone else had written them.

What stayed on top tier

A PDF that "looked corrupted" turned out to be a one-line driver difference: Puppeteer 22 and later returns a `Uint8Array` from its PDF call where Playwright returns a `Buffer`, so a plain `.toString('base64')` silently produced garbage instead of failing loudly. Nothing local caught it; it surfaced in deploy review. That kind of debugging - where the symptom lies about its own cause - is exactly what stays on the top tier.

Timeline

Step	Who	Output
1. Specs	Fable	Fixed contracts for the queue migration, worker, and render Lambda; briefs for all four builder agents
2. Parallel build	3 Sonnet builders, disjoint directories	Queue migration, worker, render Lambda - coded to the contracts, not to each other's files
3. Documentation	1 Sonnet builder	Deploy checklist and decision log
4. Seam review	Fable	Contract spot-checks; caught the anon-grant defect in its own brief
5. Security fix	Fable	Removed the anon and authenticated grant; service role only
6. Deploy review	Fable	Caught the Puppeteer Uint8Array vs Buffer bug before ship

6. Review discipline

- **Review seams, not surfaces.** Grep the contract joints - header names, param shapes, grants - instead of re-reading whole files top to bottom. The defects live where work joins.

- **Read deviations first.** They carry the real-world facts the brief got wrong: a deprecated runtime, a version pin, a name collision. They are the escalation channel, not an apology.
- **Keep parallel agents in disjoint directories.** Sequence anything that shares a file; never let two agents write the same file in the same window.
- **The brief goes in the PR description for agent-built work.** A reviewer who was not part of the delegation needs the same context the builder had - the contracts, the read-first list, what "done" meant - to review the diff on its own terms.
- **Review your own specs too.** The one real security defect in the claim-report build was in the architect's brief, not the builder's code. Seam review is not just "did the builder follow instructions" - it is "were the instructions right".

7. Setup - what each dev does

Five steps. (a) and (b) are once per machine; (c) is a one-time account link; (d) installs a background job; (e) is once per new project.

- a. **Pull any active repo.** The standard ships in-repo - `.claude/agents/`, the rules, and the Sonnet session pin are already there. Nothing separate to install for Claude Code.
- b. **Cursor users:** the twin rule is `.cursor/rules/model-delegation.mdc`. It applies automatically - no action needed.
- c. **Usage dashboard:** sign in at <https://dev-reports.customapp.co.za> with your `@customapp.co.za` Google account. Your dev row is created automatically on first sign-in.
- d. **Install the two usage extractors** (metadata only - the Claude one is POPIA-safe, no prompt text ever leaves your machine). Copy and paste into a terminal:

MACOS / LINUX

```
git clone https://github.com/Customapp-PTY-LTD/CustomappReports.git
~/Documents/GitHub/CustomappReports 2>/dev/null || git -C
~/Documents/GitHub/CustomappReports pull --ff-only
bash ~/Documents/GitHub/CustomappReports/scripts/install-usage-extractors.sh
```

WINDOWS (POWERSHELL)

```
git clone https://github.com/Customapp-PTY-LTD/CustomappReports.git
"$HOME\Documents\GitHub\CustomappReports"; if ($LASTEXITCODE -ne 0) { git -C
"$HOME\Documents\GitHub\CustomappReports" pull --ff-only }
powershell -ExecutionPolicy Bypass -File
"$HOME\Documents\GitHub\CustomappReports\scripts\install-usage-extractors.ps1"
```

Script source on GitHub: [install-usage-extractors.sh](#), [install-usage-extractors.ps1](#), [cursor-prompt-export.py](#), [claude-usage-export.py](#).

Browsers cannot safely auto-run local scripts from a click, so the copy-paste commands above are the seamless path - add `--dev <yourname>` (`-Dev <yourname>` on the PowerShell installer) to the installer call only if your dashboard username differs from your machine username.

- e. **New projects:** copy `BluePrint/model-delegation/` per its README.

8. What we measure

The dashboard plots each developer's top-tier share of total tokens, per week, against a fifteen percent target line. It is metadata only - model name, token counts, timestamps - never prompt text, for the Claude Code extractor. Cursor prompt-text collection is a separate, existing pipeline with its own notice; it is not part of this standard.

A team norm made visible, not surveillance. A week over the line is a prompt to ask why - a big architecture piece, a hard debugging session - not an automatic flag. Read it the way you would read a burn-down chart: a trend to notice, not a score to defend.

9. FAQ

When is Fable actually worth it?

Architecture decisions, anything security-adjacent, work crossing more than two components, and debugging once you are genuinely stuck - not from the first error. These are cheap in tokens and expensive in consequences if you get them wrong: the claim-report build's one real security defect (an over-broad grant) and its nastiest bug (a silent Puppeteer API difference producing corrupt PDFs) were both exactly this category.

The builder went off-spec. Now what?

Read the deviations section before anything else - it is the escalation channel, not a complaint log. A builder only deviates with a reason (a deprecated runtime, a version conflict, a name collision). Fold that fact into the next brief rather than re-running the same one and hoping.

Can I still just code on Fable?

Yes, for genuinely small things. The standard targets legwork - multi-file implementation, fan-out recon, docs from a spec - not every keystroke. A one-line fix does not need a brief.

Does this slow me down?

No - if anything the opposite. Agents run in parallel against fixed contracts instead of one session working serially through a task list. The claim-report pipeline - render Lambda, worker, migrations, and docs - landed about 3,400 lines in a single session this way.

What about Haiku?

Genuinely mechanical transforms only - changes with no judgement calls left in them. Anything touching a repo's conventions is above Haiku's bar; use Sonnet instead.

Who reviews agent code?

The same PR review as always. The only change is that the brief goes in the PR description, so a reviewer who was not part of the delegation can see the contracts and acceptance criteria the builder worked to, not just the diff.